

Healthcare Insurance Data Pipeline

Complete Implementation Guide (AWS Console)

Region: us-east-2 (Ohio)

Goal: Analyze healthcare insurance data to answer 13 business requirements

Architecture

Your Local Files (8 files)

↓

Upload to S3 (AWS Console)

↓

Glue Crawler (discovers raw data schemas)

↓

Glue ETL Job (cleans data, handles column spaces, transforms)

↓

Athena Queries (SQL analysis for 13 requirements)

Your Data Files

You have 8 data files:

- claims.json
 - disease.csv
 - group.csv
 - grpsubgrp.csv
 - hospital.csv
 - Patient_records.csv
 - subgroup.csv
 - subscriber.csv
-

STEP 1: Create S3 Bucket and Upload Data

1.1 Create S3 Bucket

1. Go to **S3 Console**: <https://s3.console.aws.amazon.com/s3/>
2. Click **Create bucket**
3. **Bucket name:** healthcare-insurance-data (must be unique - add your name/number if taken)

4. **Region:** us-east-2 (Ohio)
5. Keep all other settings as default
6. Click **Create bucket**

1.2 Create Folders

1. Click on your bucket name
2. Click **Create folder**
3. Create these folders one by one:
 - o raw/
 - o cleaned/
 - o analytics/
 - o athena-results/

1.3 Upload Your Data Files

1. Click on the **raw/** folder
2. Click **Upload**
3. Click **Add files**
4. Select all 8 data files from your computer
5. Click **Upload**
6. Wait for "Upload succeeded" message

✓ **Checkpoint:** All 8 files are now in `s3://healthcare-insurance-data/raw/`

STEP 2: Configure IAM Role for Glue

Since you already have an existing Glue role (`AWSGlueServiceRole`), we just need to add S3 permissions for this project.

2.1 Add S3 Permissions to Existing Role

1. Go to **IAM Console**: <https://console.aws.amazon.com/iam/>
2. Click **Roles** in left menu
3. Search for and click on: `AWSGlueServiceRole`
4. Click **Add permissions** → **Create inline policy**
5. Click **JSON** tab
6. Delete everything and paste:

```
{  "Version": "2012-10-17",  "Statement": [    {      "Effect": "Allow",      "Action": [        "s3:GetObject",        "s3:PutObject",        "s3:DeleteObject",        "s3:ListBucket"      ],      "Resource": [        "arn:aws:s3:::healthcare-insurance-data",        "arn:aws:s3:::healthcare-insurance-data/*"      ]    }  ]}
```
8. Click **Review policy**
9. Name: `HealthcareS3Access`

10. Click **Create policy**

✓ **Checkpoint:** Existing IAM role now has access to your healthcare S3 bucket

STEP 3: Create Glue Database

1. Go to **AWS Glue Console**: <https://console.aws.amazon.com/glue/>
2. **Verify you're in us-east-2** (top right corner)
3. Click **Databases** in left menu
4. Click **Add database**
5. **Database name:** `healthcare_db`
6. **Description:** `Healthcare insurance analytics database`
7. Click **Create database**

✓ **Checkpoint:** Database `healthcare_db` created

STEP 4: Run Glue Crawler (Discover Schemas)

4.1 Create Crawler

1. In Glue Console, click **Crawlers** in left menu
2. Click **Create crawler**

Step 1 - Set crawler properties:

- Name: `healthcare-raw-crawler`
- Click **Next**

Step 2 - Choose data sources:

- Click **Add a data source**
- Data source: **S3**
- Network connection: Leave default
- S3 path: Click **Browse** and select `healthcare-insurance-data/raw/`
- Click **Add an S3 data source**
- Click **Next**

Step 3 - Configure security settings:

- IAM role: Select `AWSGlueServiceRole` (your existing role)
- Click **Next**

Step 4 - Set output and scheduling:

- Target database: `healthcare_db`
- Crawler schedule: **On demand** (leave as is)
- Click **Next**

Step 5 - Review and create:

- Review all settings
- Click **Create crawler**

4.2 Run the Crawler

1. You'll see your crawler in the list
2. Select it (checkbox)
3. Click **Run**
4. Status will show "Running"
5. **Wait 5-10 minutes** - refresh page to see progress
6. Status changes to "Ready" when complete

4.3 Verify Tables Created

1. Click **Tables** in left menu (under Data Catalog)
2. You should see **8 tables** created:
 - claims
 - disease
 - group
 - grpsubgrp
 - hospital
 - patient_records
 - subgroup
 - subscriber
3. Click on any table (e.g., claims) to see:
 - Column names and types
 - Location (S3 path)
 - Classification (JSON/CSV)

✓ **Checkpoint:** Crawler discovered all 8 tables with schemas

STEP 5: Create Glue ETL Job (Clean & Transform Data)

5.1 Create the Job

1. In Glue Console, click **ETL jobs** in left menu
2. Click **Script editor** button
3. Click **Create**

In the dialog:

- Engine: **Spark**
- Options: **Start fresh**
- Click **Create**

5.2 Add the ETL Script

1. You'll see a script editor with some default code
2. **Select all the default code and delete it**
3. **Copy the entire script** from the artifact: **"AWS Glue ETL - Data Cleaning Script"**
4. **Paste it** into the editor

Job Script:

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql.functions import *
from pyspark.sql.types import *
# Initialize contexts
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)

# S3 bucket paths
RAW_PATH = "s3://healthcare-insurance-data/raw/"
CLEANED_PATH = "s3://healthcare-insurance-data/cleaned/"

# Helper function to clean column names
def clean_column_names(df):
    """Remove whitespace from all column names - using toDF for reliability"""
    return df.toDF(*[col_name.strip() for col_name in df.columns])

# ===== 1. CLEAN CLAIMS DATA =====
print("Cleaning claims data...")

# Read claims JSON
```

```

claims_df = spark.read.json(f"{RAW_PATH}claims.json")
claims_df = clean_column_names(claims_df)

# Clean claims
claims_cleaned = claims_df \
    .withColumn("claim_id", col("claim_id").cast(IntegerType())) \
    .withColumn("patient_id", col("patient_id").cast(IntegerType())) \
    .withColumn("claim_amount",
        when(col("claim_amount").isNotNull(),
            col("claim_amount").cast(IntegerType()))
        .otherwise(0)) \
    .withColumn("claim_date", to_date(col("claim_date"), "yyyy-MM-dd")) \
    .withColumn("Claim_Or_Rejected",
        when(col("Claim_Or_Rejected") == "NaN", None)
        .otherwise(col("Claim_Or_Rejected"))) \
    .withColumn("claim_status",
        when(col("Claim_Or_Rejected") == "Y", "Rejected")
        .when(col("Claim_Or_Rejected") == "N", "Approved")
        .otherwise("Pending")) \
    .dropDuplicates(["claim_id"])

# Write to S3 as Parquet (partitioned by status for better performance)
claims_cleaned.write \
    .mode("overwrite") \
    .partitionBy("claim_status") \
    .parquet(f"{CLEANED_PATH}claims/")

print(f"Claims cleaned: {claims_cleaned.count()} records")

# ===== 2. CLEAN DISEASE DATA =====
print("Cleaning disease data...")

disease_df = spark.read.option("header",
    "true").csv(f"{RAW_PATH}disease.csv")
disease_df = clean_column_names(disease_df)

# Debug: Print actual column names to see the issue
print("Disease columns:", disease_df.columns)

# Rename columns explicitly to avoid whitespace issues
disease_df = disease_df.toDF(*[col_name.strip() for col_name in
    disease_df.columns])

disease_cleaned = disease_df \
    .withColumn("SubGrpID", trim(col("SubGrpID"))) \

```

```

.withColumn("Disease_ID", trim(col("Disease_ID"))) \
.withColumn("Disease_name", trim(col("Disease_name"))) \
.dropDuplicates(["Disease_ID"])

disease_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}disease/")
print(f"Disease data cleaned: {disease_cleaned.count()} records")

# ===== 3. CLEAN GROUP DATA =====
print("Cleaning group data...")

group_df = spark.read.option("header", "true").csv(f"{RAW_PATH}group.csv")
group_df = clean_column_names(group_df)

group_cleaned = group_df \
.withColumn("premium_written", col("premium_written").cast(IntegerType())) \
.withColumn("zipcode", col("zipcode").cast(IntegerType())) \
.withColumn("year", col("year").cast(IntegerType())) \
.withColumn("Grp_Type",
             when(col("Grp_Type") == "Govt.", "Government")
             .otherwise("Private")) \
.dropDuplicates(["Grp_Id"])

group_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}group/")
print(f"Group data cleaned: {group_cleaned.count()} records")

# ===== 4. CLEAN HOSPITAL DATA =====
print("Cleaning hospital data...")

hospital_df = spark.read.option("header",
"true").csv(f"{RAW_PATH}hospital.csv")
hospital_df = clean_column_names(hospital_df)
hospital_cleaned = hospital_df \
.withColumn("state",
             when(col("state") == "NaN", None)
             .otherwise(col("state"))) \
.dropDuplicates(["Hospital_id"])

hospital_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}hospital/")
print(f"Hospital data cleaned: {hospital_cleaned.count()} records")

# ===== 5. CLEAN PATIENT RECORDS =====
print("Cleaning patient records...")

```

```

patient_df = spark.read.option("header",
"true").csv(f"{RAW_PATH}Patient_records.csv")
patient_df = clean_column_names(patient_df)

patient_cleaned = patient_df \
    .withColumn("Patient_id", col("Patient_id").cast(IntegerType())) \
    .withColumn("patient_birth_date", to_date(col("patient_birth_date"), "yyyy-MM-
dd")) \
    .withColumn("age",
        floor(datediff(current_date(), col("patient_birth_date")) / 365.25)) \
    .withColumn("Patient_name",
        when(col("Patient_name").isNull(), "Unknown")
        .otherwise(col("Patient_name"))) \
    .dropDuplicates(["Patient_id"])

patient_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}patient_rec
ords/")
print(f"Patient records cleaned: {patient_cleaned.count()} records")

```

```

# ===== 6. CLEAN SUBSCRIBER DATA =====
print("Cleaning subscriber data...")

```

```

subscriber_df = spark.read.option("header",
"true").csv(f"{RAW_PATH}subscriber.csv")
subscriber_df = clean_column_names(subscriber_df)

# Fix the "sub_id" column name (has space in middle)
if "sub_id" in subscriber_df.columns:
    subscriber_df = subscriber_df.withColumnRenamed("sub_id", "sub_id")

subscriber_cleaned = subscriber_df \
    .withColumn("Birth_date", to_date(col("Birth_date"), "yyyy-MM-dd")) \
    .withColumn("eff_date", to_date(col("eff_date"), "yyyy-MM-dd")) \
    .withColumn("term_date", to_date(col("term_date"), "yyyy-MM-dd")) \
    .withColumn("age",
        floor(datediff(current_date(), col("Birth_date")) / 365.25)) \
    .withColumn("Elig_ind",
        when(col("Elig_ind").isNull(), "N")
        .otherwise(col("Elig_ind"))) \
    .withColumn("Subgrp_id",
        when(col("Subgrp_id").isNull(), "Unknown")
        .otherwise(col("Subgrp_id"))) \
    .dropDuplicates(["sub_id"])

```

```
subscriber_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}subscriber/")
print(f"Subscriber data cleaned: {subscriber_cleaned.count()} records")
```

```
# ===== 7. CLEAN SUBGROUP DATA =====
print("Cleaning subgroup data...")
```

```
subgroup_df = spark.read.option("header",
"true").csv(f"{RAW_PATH}subgroup.csv")
subgroup_df = clean_column_names(subgroup_df)
```

```
subgroup_cleaned = subgroup_df \
    .withColumn("Monthly_Premium",
col("Monthly_Premium").cast(IntegerType())) \
    .dropDuplicates(["SubGrp_id"])
```

```
subgroup_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}subgroup/")
print(f"Subgroup data cleaned: {subgroup_cleaned.count()} records")
```

```
# ===== 8. CLEAN GROUP-SUBGROUP MAPPING =====
print("Cleaning group-subgroup mapping...")
```

```
grpsubgrp_df = spark.read.option("header",
"true").csv(f"{RAW_PATH}grpsubgrp.csv")
grpsubgrp_df = clean_column_names(grpsubgrp_df)
```

```
grpsubgrp_cleaned = grpsubgrp_df.dropDuplicates()
```

```
grpsubgrp_cleaned.write.mode("overwrite").parquet(f"{CLEANED_PATH}grpsubgrp/")
print(f"Group-Subgroup mapping cleaned: {grpsubgrp_cleaned.count()} records")
```

```
# ===== 9. CREATE ANALYTICS TABLES =====
print("Creating analytics tables...")
```

```
# Create comprehensive view joining all tables
analytics_df = claims_cleaned \
    .join(patient_cleaned, "patient_id", "left") \
    .join(hospital_cleaned, "Hospital_id", "left") \
    .join(disease_cleaned,
        claims_cleaned.disease_name == disease_cleaned.Disease_name,
        "left") \
```

```

.join(subscriber_cleaned,
      claims_cleaned.SUB_ID == subscriber_cleaned.sub_id,
      "left") \
.join(subgroup_cleaned,
      subscriber_cleaned.Subgrp_id == subgroup_cleaned.SubGrp_id,
      "left") \
.select(
  claims_cleaned["*"],
  patient_cleaned["Patient_name"],
  patient_cleaned["patient_gender"],
  patient_cleaned["age"].alias("patient_age"),
  patient_cleaned["city"].alias("patient_city"),
  hospital_cleaned["Hospital_name"],
  hospital_cleaned["state"].alias("hospital_state"),
  disease_cleaned["SubGrpID"],
  subgroup_cleaned["SubGrp_Name"],
  subgroup_cleaned["Monthly_Premium"],
  subscriber_cleaned["Elig_ind"]
)

analytics_df.write \
  .mode("overwrite") \
  .partitionBy("claim_status") \
  .parquet(f"{CLEANED_PATH}analytics_consolidated/")

print(f"Analytics table created: {analytics_df.count()} records")

# Data quality checks
print("\n=== DATA QUALITY SUMMARY ===")
print(f"Total Claims: {claims_cleaned.count()}")
print(f"Rejected Claims: {claims_cleaned.filter(col('claim_status') == 'Rejected').count()}")
print(f"Approved Claims: {claims_cleaned.filter(col('claim_status') == 'Approved').count()}")
print(f"Pending Claims: {claims_cleaned.filter(col('claim_status') == 'Pending').count()}")
print(f"Total Claim Amount: {claims_cleaned.agg(sum('claim_amount')).collect()[0][0]}")

job.commit()
print("\nETL Job completed successfully!")

```

5.3 Configure Job Settings

1. Click **Job details** tab (at the top)
2. Fill in these settings:

Basic properties:

- Name: healthcare-data-cleaning
- IAM Role: AWSGlueServiceRole (your existing role)
- Type: **Spark**
- Glue version: **Glue 4.0**
- Language: **Python 3**

Job resource configuration:

- Worker type: **G.1X** (most cost-effective)
- Number of workers: **2**

Advanced properties:

- Job timeout: **120 minutes**
- Number of retries: **0**

3. Click **Save** (top right)

5.4 Run the ETL Job

1. Click **Run** button (top right)
2. You'll see "Job run started" message
3. Click **Runs** tab to monitor progress
4. Status shows: Starting → Running → Succeeded
5. **This takes 10-15 minutes** - be patient!

What the job does:

- Reads raw data from S3
- Cleans column names (removes spaces)
- Fixes data types
- Handles NaN values
- Removes duplicates
- Creates cleaned Parquet files
- Saves to `s3://healthcare-insurance-data/cleaned/`

5.5 Verify Job Succeeded

1. When status shows **Succeeded** ✓
2. Go back to **S3 Console**
3. Navigate to your bucket → `cleaned/` folder
4. You should see **9 folders**:
 - `claims/`
 - `disease/`

- group/
 - grpsubgrp/
 - hospital/
 - patient_records/
 - subscriber/
 - subgroup/
 - analytics_consolidated/
5. Click into any folder - you'll see Parquet files

✓ **Checkpoint:** Data cleaned and transformed successfully!

STEP 6: Setup Athena for Queries

6.1 Configure Athena

1. Go to **Athena Console**: <https://console.aws.amazon.com/athena/>
2. **Verify you're in us-east-2** region
3. You may see a "Before you run your first query" message
4. Click **Settings** (or **Manage** if you see it)
5. Click **Manage** next to "Query result location"
6. Enter: `s3://healthcare-insurance-data/athena-results/`
7. Click **Save**

6.2 Create Crawler for Cleaned Data

We need to tell Glue about our cleaned Parquet files.

1. Go back to **Glue Console** → **Crawlers**
2. Click **Create crawler**

Step 1:

- Name: `healthcare-cleaned-crawler`
- Click **Next**

Step 2:

- Click **Add a data source**
- S3 path: Browse to `healthcare-insurance-data/cleaned/`
- Click **Add an S3 data source**
- Click **Next**

Step 3:

- IAM role: `AWSGlueServiceRole` (your existing role)
- Click **Next**

Step 4:

- Target database: `healthcare_db`
- Table name prefix: `cleaned_` (optional but helpful)
- Click **Next**

Step 5:

- Click **Create crawler**
3. **Run the crawler:**
 - Select it and click **Run**
 - Wait 5 minutes
 4. **Verify:**
 - Go to **Tables**
 - You should now see cleaned tables like:
 - `cleaned_claims`
 - `cleaned_disease`
 - `cleaned_analytics_consolidated`
 - etc.

✓ **Checkpoint:** Athena can now query the cleaned data!

STEP 7: Run Analytics Queries in Athena

7.1 Select Database

1. In Athena Console
2. Left panel: **Database** dropdown
3. Select: `healthcare_db`
4. You'll see all tables listed below

7.2 Test Query

Copy and paste this test query:

```
SELECT * FROM cleaned_claims LIMIT 10;
```

Click **Run**

You should see 10 rows of claims data!

7.3 Run Your 13 Required Queries

Now use the queries from the artifact: "Athena SQL Queries - All Requirements"

Example queries:

1. Disease with maximum claims:

```
SELECT
    disease_name,
    COUNT(*) as total_claims,
    SUM(claim_amount) as total_claim_amount
FROM healthcare_db.claims
WHERE claim_status != 'Rejected'
GROUP BY disease_name
ORDER BY total_claims DESC
LIMIT 1;
```

2. Subscribers under 30 and age subgroups:

```
SELECT
    CASE
        WHEN age < 18 THEN '0-17 (Minor)'
        WHEN age BETWEEN 18 AND 30 THEN '18-30 (Young Adult)'
        WHEN age BETWEEN 31 AND 45 THEN '31-45 (Adult)'
        WHEN age BETWEEN 46 AND 60 THEN '46-60 (Middle Age)'
        ELSE '60+ (Senior)'
    END as age_group,
    COUNT(*) as patient_count,
    AVG(age) as avg_age
FROM healthcare_db.patient_records
WHERE age IS NOT NULL
GROUP BY
    CASE
        WHEN age < 18 THEN '0-17 (Minor)'
        WHEN age BETWEEN 18 AND 30 THEN '18-30 (Young Adult)'
        WHEN age BETWEEN 31 AND 45 THEN '31-45 (Adult)'
        WHEN age BETWEEN 46 AND 60 THEN '46-60 (Middle Age)'
        ELSE '60+ (Senior)'
    END
ORDER BY avg_age;
```

3. Hospital serving most patients:

```
SELECT
    h.hospital_id,
    h.hospital_name,
    COUNT(p.patient_id) AS patient_count
FROM healthcare_db.hospital h
JOIN healthcare_db.patient_records p
    ON h.hospital_id = p.hospital_id
```

```

GROUP BY h.hospital_id, h.hospital_name
ORDER BY patient_count DESC
LIMIT 3;

```

4. Total rejected claims:

```

SELECT
    claim_status,
    COUNT(*) as total_rejected_claims,
    SUM(claim_amount) as total_rejected_amount,
    AVG(claim_amount) as avg_rejected_amount
FROM healthcare_db.claims
WHERE claim_status = 'Rejected'
GROUP BY claim_status;

```

5. Subgroups who subscribe most number of times:

```

SELECT
    sg.subgrp_id,
    sg.subgrp_name,
    sg.monthly_premium,
    COUNT(s.sub_id) as subscriber_count
FROM healthcare_db.subgroup sg
LEFT JOIN healthcare_db.subscriber s ON sg.subgrp_id = s.subgrp_id
GROUP BY sg.subgrp_id, sg.subgrp_name, sg.monthly_premium
ORDER BY subscriber_count DESC;

```

6. Cities with most claims:

```

SELECT
    p.city,
    COUNT(c.claim_id) as total_claims,
    SUM(c.claim_amount) as total_claim_amount,
    AVG(c.claim_amount) as avg_claim_amount
FROM healthcare_db.claims c
JOIN healthcare_db.patient_records p ON c.patient_id = p.patient_id
GROUP BY p.city
ORDER BY total_claims DESC
LIMIT 5;

```

7. Government vs Private:

```

SELECT
    g.grp_type,
    COUNT(DISTINCT g.grp_id) as total_groups,
    SUM(g.premium_written) as total_premium,
    AVG(g.premium_written) as avg_premium_per_group,
    COUNT(DISTINCT gs.subgrp_id) as total_subgroups
FROM healthcare_db."group" g
LEFT JOIN healthcare_db.grpsubgrp gs ON g.grp_id = gs.grp_id
GROUP BY g.grp_type;

```

8. Average Monthly Premium

```

SELECT
    ROUND(AVG(sg.monthly_premium), 2) as avg_monthly_premium,
    MIN(sg.monthly_premium) as min_premium,
    MAX(sg.monthly_premium) as max_premium,
    COUNT(DISTINCT s.sub_id) as total_subscribers
FROM healthcare_db.subscriber s
JOIN healthcare_db.subgroup sg ON s.subgrp_id = sg.subgrp_id;

```

9. Most Profitable group:

```

WITH group_revenue AS (
    SELECT g.grp_id,
           g.grp_name,
           g.grp_type,
           SUM(
               sg.monthly_premium * date_diff(
                   'month',
                   s.eff_date,
                   COALESCE(s.term_date, CURRENT_DATE)
               )
           ) as total_premium_collected
    FROM healthcare_db."group" g
    JOIN healthcare_db.grpsubgrp gs ON g.grp_id = gs.grp_id
    JOIN healthcare_db.subscriber s ON gs.subgrp_id =
s.subgrp_id
    JOIN healthcare_db.subgroup sg ON s.subgrp_id =
sg.subgrp_id
    GROUP BY g.grp_id,
             g.grp_name,
             g.grp_type
),
group_claims AS (
    SELECT g.grp_id,
           SUM(c.claim_amount) as total_claims_paid
    FROM healthcare_db."group" g
    JOIN healthcare_db.grpsubgrp gs ON g.grp_id = gs.grp_id
    JOIN healthcare_db.subscriber s ON gs.subgrp_id =
s.subgrp_id
    JOIN healthcare_db.claims c ON s.sub_id = c.sub_id
    WHERE c.claim_status = 'Approved'
    GROUP BY g.grp_id
)
SELECT r.grp_id,
       r.grp_name,
       r.grp_type,
       r.total_premium_collected,
       COALESCE(c.total_claims_paid, 0) as total_claims_paid,
       (
           r.total_premium_collected - COALESCE(c.total_claims_paid,
0)
       ) as profit
FROM group_revenue r
LEFT JOIN group_claims c ON r.grp_id = c.grp_id

```

```
ORDER BY profit DESC
LIMIT 5;
```

10. Cancer patients under 18:

```
SELECT
    p.patient_id,
    p.patient_name,
    p.patient_gender,
    p.age,
    p.disease_name,
    p.city,
    h.hospital_name
FROM healthcare_db.patient_records p
JOIN healthcare_db.hospital h ON p.hospital_id = h.hospital_id
WHERE p.age < 18
    AND LOWER(p.disease_name) LIKE '%cancer%'
ORDER BY p.age;
```

11. Cashless insurance and charges greater than or equal to Rs. 50000:

```
SELECT
    c.patient_id,
    p.patient_name,
    p.patient_gender,
    p.age,
    c.disease_name,
    c.claim_amount,
    c.claim_date,
    s.elig_ind as cashless_eligible
FROM healthcare_db.claims c
JOIN healthcare_db.patient_records p ON c.patient_id = p.patient_id
JOIN healthcare_db.subscriber s ON c.sub_id = s.sub_id
WHERE s.elig_ind = 'Y' -- Eligible for cashless
    AND c.claim_amount >= 50000
    AND c.claim_status != 'Rejected'
ORDER BY c.claim_amount DESC
LIMIT 10;
```

12. Female patients over 40 and undergone knee surgery:

```
SELECT
    p.patient_id,
    p.patient_name,
    p.patient_gender,
    p.age,
    p.disease_name,
    c.claim_date,
    c.claim_amount,
    h.hospital_name
FROM healthcare_db.patient_records p
JOIN healthcare_db.claims c ON p.patient_id = c.patient_id
JOIN healthcare_db.hospital h ON p.hospital_id = h.hospital_id
WHERE p.patient_gender = 'Female'
    AND p.age > 40
    AND (LOWER(p.disease_name) LIKE '%knee%'
        OR LOWER(c.claim_type) LIKE '%surgery%'
        OR LOWER(p.disease_name) LIKE '%arthritis%'
        OR LOWER(p.disease_name) LIKE '%fracture%')
```

```
AND c.claim_date >= DATE_ADD('year', -1, CURRENT_DATE)
ORDER BY c.claim_date DESC;
```

7.4 Save Query Results

- Athena automatically saves results to: `s3://healthcare-insurance-data/athena-results/`
- You can download results as CSV
- Click **Download results** button after each query

✓ **Checkpoint:** All 13 requirements answered with SQL queries!

STEP 8: Create QuickSight Dashboards

8.1 Setup QuickSight (If First Time)

1. Go to **QuickSight Console**: <https://quicksight.aws.amazon.com/>
2. If not signed up yet:
 - Click **Sign up for QuickSight**
 - Choose **Standard Edition** (\$9/user/month)
 - Select region: **us-east-2**
 - Give QuickSight access to S3 (select your bucket)

8.2 Connect QuickSight to Athena

1. In QuickSight, click **Datasets** (left menu)
2. Click **New dataset**
3. Select **Athena** as data source
4. Data source name: `healthcare-analytics`
5. Click **Create data source**
6. Select database: `healthcare_db`

8.3 Dashboard 1: Claims Overview

Create Dataset:

1. Select table: `cleaned_analytics_consolidated`
2. Click **Select**
3. Choose: **Import to SPICE** (for faster performance)
4. Click **Visualize**

Create Visualizations:

1. **Claim Amount by Disease (Horizontal Bar Chart)**

- Visual type: Horizontal bar chart
 - Y-axis: `disease_name`
 - Value: `claim_amount` (Sum)
 - Sort: Descending by total claim amount
 - Shows diseases like Stroke, Galactosemia, Drug consumption as top claimants
2. **Claim Amount Distribution (Histogram)**
 - Visual type: Histogram
 - Field: `claim_amount`
 - Bins: Auto (showing distribution from ~12K to 193K)
 - Shows frequency distribution of claim amounts
 3. **Claim Status (Pie Chart)**
 - Visual type: Pie chart
 - Group: `claim_status`
 - Shows three categories: Rejected, Pending, Approved
 - Color-coded: Blue (Rejected), Gray (Pending), Orange (Approved)
 4. **Claim Amount by Gender (Bar Chart)**
 - Visual type: Vertical bar chart
 - X-axis: `patient_gender`
 - Value: `claim_amount` (Sum)
 - Compares total claim amounts between Male and Female patients

Save Dashboard:

- Click **Share** → **Publish dashboard**
- Name: `Claims Overview`
- Click **Publish dashboard**

8.4 Dashboard 2: Premiums by Group

Create New Analysis:

1. Click **Analyses** (left menu)
2. Click **New analysis**
3. Use dataset with `cleaned_group` joined with related tables
4. Click **Visualize**

Create Visualizations:

1. **Government vs Private (Horizontal Bar Chart)**
 - Visual type: Horizontal bar chart
 - Y-axis: `grp_type` (Government, Private)
 - Value: `premium_written` (Sum)
 - Shows Private insurers collect ~3.5M vs Government ~0.5M
 - Clear comparison of market dominance
2. **Premiums by Name (Horizontal Bar Chart)**
 - Visual type: Horizontal bar chart

- Y-axis: grp_name (Insurance company names)
- Value: premium_written (Sum)
- Sort: Descending
- Shows top performers:
 - Cholamandalam MS General Insurance (~99K)
 - IndiaFirst Life Insurance
 - Raheja QBE General Insurance
 - IDBI Federal Life Insurance
 - And more companies

Save Dashboard:

- Click **Share** → **Publish dashboard**
- Name: Premiums by Group
- Click **Publish dashboard**

8.5 Share Dashboards

1. Open each dashboard
2. Click **Share** (top right)
3. Click **Share dashboard**
4. Enter email addresses of stakeholders
5. Set permissions (View only recommended)
6. Click **Share**

✓ **Checkpoint:** Two interactive dashboards created and published!

Your Completed Dashboards

Dashboard 1: Claims Overview

Visual Insights:

- **Claim Amount by Disease:** Stroke has highest total claims (~400K), followed by Galactosemia and Drug consumption
- **Claim Distribution:** Most claims fall between 12K-193K range with varying frequencies
- **Claim Status:** Shows proportion of Rejected, Pending, and Approved claims
- **Gender Analysis:** Compares claim amounts between Male (~4M) and Female patients

Key Findings:

- Stroke-related claims are most expensive
- Clear view of approval/rejection patterns

- Gender-based claim amount comparison visible

Dashboard 2: Premiums by Group

Visual Insights:

- **Government vs Private:** Private insurers dominate with ~3.5M in premiums vs ~0.5M Government
- **Top Performers:**
 - Cholamandalam MS General Insurance leads (~99K premium)
 - IndiaFirst Life Insurance
 - Raheja QBE General Insurance
 - IDBI Federal Life Insurance
 - Plus 8+ other major players shown

Key Findings:

- Private insurance market is 7x larger than government
 - Clear market leaders identified
 - Easy to compare performance across companies
-

Troubleshooting

Problem: Crawler shows 0 tables

Solution:

- Check files uploaded to S3 correctly
- Verify IAM role has S3 permissions
- Check you selected correct S3 path

Problem: Glue job fails

Solution:

- Go to CloudWatch Logs
- Navigate to `/aws-glue/jobs/logs-v2/`
- Find your job logs
- Look for error messages
- Common issue: IAM permissions

Problem: Athena query returns "Table not found"

Solution:

- Run: `SHOW TABLES;` to see available tables
- Make sure you ran the second crawler for cleaned data
- Check database is selected in dropdown

Problem: Query returns "Column not found"

Solution:

- Check table schema: `DESCRIBE table_name;`
 - Make sure you're querying `cleaned_` tables
 - Column names are case-sensitive
-

Summary - What You Accomplished

- ✓ **Step 1:** Uploaded 8 data files to S3
- ✓ **Step 2:** Configured IAM role with S3 permissions
- ✓ **Step 3:** Created Glue database
- ✓ **Step 4:** Ran crawler to discover schemas
- ✓ **Step 5:** Ran ETL job to clean and transform data
- ✓ **Step 6:** Configured Athena for queries
- ✓ **Step 7:** Answered all 13 business requirements
- ✓ **Step 8:** Created 2 interactive QuickSight dashboards

Your Deliverables:

- ✓ Claims Overview Dashboard
- ✓ Premiums by Group Dashboard
- ✓ All 13 SQL query results
- ✓ Complete data pipeline architecture

Cost Estimate

For this complete project (one-time setup + 1 month):

- **S3 Storage:** ~\$0.25/month (small dataset)
- **Glue Crawler:** ~\$0.44 (one-time run × 2)
- **Glue ETL Job:** ~\$0.44 (one-time run)
- **Athena Queries:** ~\$0.05-0.10 (based on usage)
- **QuickSight:** \$9/month (Standard Edition, 1 user)

Total: ~\$10-11/month (mostly QuickSight subscription)

Note: You can cancel QuickSight subscription anytime if not needed ongoing

Next Steps (Optional)

If you want to continue:

- Schedule ETL job to run daily (for new data)
 - Create QuickSight dashboards for visualization
 - Set up CloudWatch alarms for monitoring
 - Add data quality checks
-